

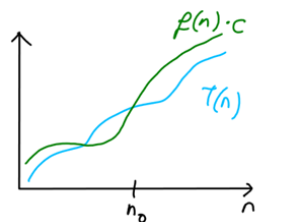
❖ **What is the goal of asymptotic notation? Demonstrate and explain the asymptotic upper bound.**

The goal is to analyze an algorithm's running time as a function of input size 'n' while ignoring hardware dependencies and implementation details.

- **Abstract Measurement:** Focus on the high-level pseudo-code rather than low-level machine instructions.
- **Ignore Constants & Lower-Order Terms:** Running time is expressed in terms of dominant growth (e.g., $O(n^2)$ for $g(n)=1.62n^2+3.5n+8$)
- **Hardware Independence:** The analysis remains valid regardless of specific hardware optimizations.

Big-O Notation (O) – Asymptotic Upper Bound

Purpose: Big-O notation provides an upper limit on the growth of an algorithm's running time. It describes the worst-case scenario, ensuring the algorithm does not grow faster than a certain rate.



Definition: A function $T(n)$ is $O(f(n))$ if there exist constants c and n_0 such that:

$$T(n) \leq c \cdot f(n)$$

- c is a constant that does **not** depend on n .
- n_0 is the point from which the upper bound holds.

Example:

Suppose $T(n) = 7n^2 + 5$.

To prove that $T(n)$ is $O(n^3)$:

1. Check if $T(n) \leq c \cdot n^3$ for some constant c .
2. Rewrite:

$$7n^2 + 5 \leq 7n^2 + 5n^2 = 12n^2$$

Since $12n^2 \leq 12n^3$ for large n , we can say:

$$T(n) \leq 12n^3, \quad \text{for } n \geq 1, c = 12.$$

3. So, $T(n) = O(n^3)$.

However, since a looser upper bound is still valid, we could also write $T(n) = O(n^4)$, but it is less precise.

Asymptotic Lower Bound (Ω - Omega Notation)

1. Motivation:

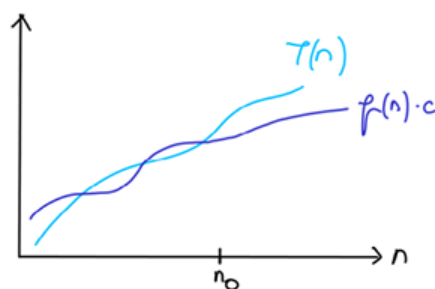
- We want to find a function $f(n)$ that acts as a **lower bound** for an algorithm's running time $T(n)$.
- This helps us understand **the minimum time required for any algorithm** solving the problem.

2. Definition:

- $T(n) = \Omega(f(n))$ means that there exist constants $c > 0$ and n_0 such that:

$$T(n) \geq c \cdot f(n), \quad \forall n \geq n_0$$

- This ensures that for large enough n , the function $f(n)$ always stays below or equal to $T(n)$.



3. Example:

Let $T(n) = 7n^2 + 5$, and check if $T(n) = \Omega(n^2)$.

Proof:

$$T(n) = 7n^2 + 5 \geq 7n^2 \quad \text{for all } n \geq 0$$

Here, $c = 7$ and $n_0 = 0$, which satisfies the condition.

Thus, $T(n) = \Omega(n^2)$.

Asymptotically Tight Bound (Θ - Theta Notation)

1. Motivation:

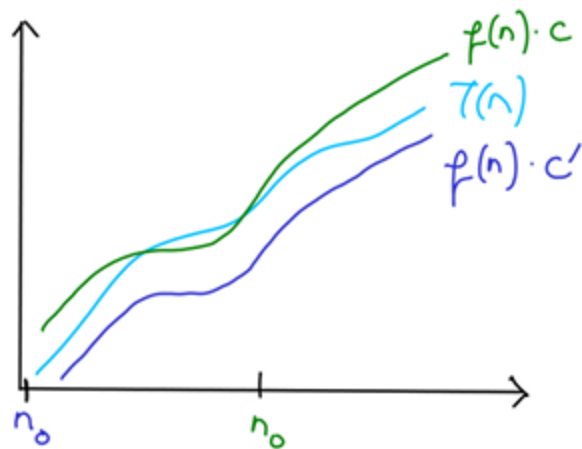
- We want a **precise description of an algorithm's worst-case running time**.
- Θ notation gives both an **upper (O) and a lower (Ω) bound**, making it a **tight bound**.

2. Definition:

- $T(n) = \Theta(f(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that:

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n), \quad \forall n \geq n_0$$

- This means $f(n)$ gives an **accurate estimate of $T(n)$** as n grows large.



3. Example:

Let $T(n) = 7n^2 + 5$, and find $\Theta(n^2)$.

✓ Step 1: Show $T(n) = O(n^2)$

$$T(n) = 7n^2 + 5 \leq 12n^2 \text{ for } n \geq 1$$

Here, $c_2 = 12$.

✓ Step 2: Show $T(n) = \Omega(n^2)$

$$T(n) = 7n^2 + 5 \geq n^2 \text{ for all } n \geq 1$$

Here, $c_1 = 1$.

Since both are true, $T(n) = \Theta(n^2)$. ✓

Properties of Asymptotic Growth Rates

Motivation

To derive an asymptotically tight bound, we analyze the limit of a function $f(n)$ representing the worst-case runtime of an algorithm as $n \rightarrow \infty$.

Theorem

If for two functions $f(n)$ and $g(n)$,

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c, \quad c > 0,$$

then $f(n) = \Theta(g(n))$.

Proof

Using the definition of Θ :

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

From the given limit, there exists n_0 such that:

$$\frac{1}{2}c \leq \frac{f(n)}{g(n)} \leq 2c, \quad \forall n \geq n_0.$$

This implies:

- $f(n) \leq 2cg(n) \Rightarrow f(n) = O(g(n)).$
 - $\frac{c}{2}g(n) \leq f(n) \Rightarrow f(n) = \Omega(g(n)).$
- Thus, $f(n) = \Theta(g(n)).$

Lemmas

Transitivity of Asymptotic Notations

If $f(n)$ is asymptotically bounded by $g(n)$ and $g(n)$ is bounded by $h(n)$, then:

- $f = O(g) \text{ and } g = O(h) \Rightarrow f = O(h).$
- $f = \Omega(g) \text{ and } g = \Omega(h) \Rightarrow f = \Omega(h).$
- $f = \Theta(g) \text{ and } g = \Theta(h) \Rightarrow f = \Theta(h).$

Additivity of Big-O

If $f(n) = O(h(n))$ and $g(n) = O(h(n))$, then:

$$f(n) + g(n) = O(h(n)).$$

Consequence of Additivity

If $g = O(f)$, then:

$$f(n) + g(n) = O(f(n)).$$

Also, since $f(n) + g(n) \geq f(n)$, we get:

$$f(n) + g(n) = \Theta(f(n)).$$

These properties help simplify algorithm complexity analysis.

Properties of Asymptotic Growth Rates

Transitivity: If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Additivity: If $f(n) = O(h(n))$ and $g(n) = O(h(n))$, then $f(n) + g(n) = O(h(n))$.

Polynomial Rule: If $f(n)$ is a polynomial of degree d , then $f(n) = O(n^d)$.

Logarithm Rule: $\log_b(n) = O(n^r)$ for any $r > 0$.

Exponential Rule: $n^d = O(r^n)$ for $d > 0$ and $r > 1$.

2.5 Asymptotic Bounds for Common Functions

Polynomial Functions

- A polynomial function $F(x)$ is of the form:

$$F(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

where $a_n \neq 0$ and n is the degree.

Lemma:

- If $F(x)$ is a polynomial of degree d , then $F(x) = O(n^d)$.
- The highest-degree term determines the asymptotic growth.

Logarithmic Functions

Lemma:

- For any base $b > 1$ and any $r > 0$:

$$\log_b(n) = O(n^r)$$

- Logarithmic functions grow slower than any polynomial function.
- Any two logarithm bases differ by a constant factor:

$$\log_b(n) = \frac{\log_k(n)}{\log_k(b)}$$

Exponential Functions

- An exponential function is of the form:

$$F(x) = r^x, \quad r > 1$$

Lemma:

- For any $d > 0$ and $r > 1$:

$$n^d = O(r^n)$$

- Exponential functions grow faster than polynomials.

Warning:

- Logarithmic functions in different bases are asymptotically the same:

$$\log_b(n) = \Theta(\log_k(n))$$

- But exponential functions in different bases are **not** the same:

$$b^n \neq \Theta(k^n), \quad \text{for } b > k > 1$$

6. Common Algorithm Complexities

Complexity	Example Algorithm	Graph Shape
$O(1)$ (Constant Time)	Accessing an element in an array	Flat line
$O(\log n)$ (Logarithmic Time)	Binary search	Grows slowly
$O(n)$ (Linear Time)	Merging two sorted lists	Straight line
$O(n \log n)$ (Log-Linear Time)	Merge Sort, QuickSort (average case)	Slight curve
$O(n^2)$ (Quadratic Time)	Bubble Sort, Insertion Sort	Steeper curve
$O(n^3)$ (Cubic Time)	Matrix multiplication (naïve approach)	Even steeper
$O(2^n)$ (Exponential Time)	Solving the traveling salesman problem (brute force)	Rapid growth
$O(n!)$ (Factorial Time)	Solving the n-queens problem (brute force)	Extremely rapid growth

